

Eccezioni

Errori Java

Come per qualsiasi linguaggio di programmazione, anche per il linguaggio Java uno dei principali problemi riscontrati dai programmatori è la presenza di errori; la chiave è **trovarli** e poi **risolverli**.

In Java esistono tre principali categorie di errori:

1. **errori a tempo di compilazione**, individuati dal compilatore, che impediscono l'esecuzione del codice. Alcuni esempi sono la mancanza di un punto e virgola ';' alla fine di una istruzione, l'utilizzo di una variabile non dichiarata oppure un mismatch (una non corrispondenza) tra il tipo di una variabile ed il suo valore (per esempio proviamo ad assegnare una stringa ad una variabile intera);
2. **errori a runtime**, che si verificano mentre il codice è in esecuzione e spesso causano un crash dell'applicazione, come cercare di dividere per zero o accedere ad un array usando un indice non valido;
3. **errori logici**, spesso i più difficili da individuare: il codice esegue, ma non restituisce il risultato atteso.

Tipicamente, quando si verifica un errore Java interrompe l'esecuzione del programma e segnala un errore (tranne per gli errori logici). Più tecnicamente, si dice che Java *ha lanciato* una **eccezione**.

Try-catch

Il linguaggio Java offre la possibilità di gestire a tempo di esecuzione (runtime) le eccezioni che vengono lanciate, così da evitare di mandare in crash l'applicazione.

Per farlo viene usata una particolare struttura, utilizzando due parole chiave:

- **try**, che consente di definire un blocco di codice che verrà testato a runtime per controllare se si presentano errori (eccezioni);
- **catch**, che consente di definire un blocco di codice da eseguire qualora un certo tipo di errore si dovesse verificare.

N.B.: le parole chiave try e catch sono sempre presenti in coppia (non possiamo usare una senza l'altra).

Sintassi:

```
try{  
    //blocco di istruzioni da testare  
}catch(Exception e){  
    //blocco di istruzioni per gestire l'errore  
}
```

Esempio:

```
public class Main {  
    public static void main(String[ ] args) {  
        int[] myNumbers = {1, 2, 3};  
        System.out.println(myNumbers[10]); // errore!  
        //altre istruzioni...  
    }  
}
```

Il codice sopra riportato genererà il seguente errore:

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 10  
    at Main.main(Main.java:4)
```

Ovvero all'interno della classe Main.java, nel metodo main(), alla riga 4, si cerca di accedere all'array *myNumbers* alla posizione 10, che però non esiste!

L'applicazione a questo punto termina l'esecuzione non appena l'errore viene sollevato.

Possiamo evitare che ciò accada gestendo l'eccezione e segnalando che si è verificato un errore:

```
public class Main {  
    public static void main(String[ ] args) {  
        try {  
            int[] myNumbers = {1, 2, 3};  
            System.out.println(myNumbers[10]);  
        } catch (Exception e) {  
            System.out.println("Qualcosa è andato storto");  
        }  
    }  
}
```

In questo modo il programma non verrà bloccato, ma segnalerà l'errore e poi proseguirà la sua esecuzione.

Quello che tipicamente viene fatto è richiamare il metodo *getMessage()* dell'oggetto di tipo eccezione, che ci consente di far stampare il dettaglio dell'errore.

```

public class Main {
    public static void main(String[] args) {
        try {
            int[] myNumbers = {1, 2, 3};
            System.out.println(myNumbers[10]);
        } catch (Exception e) {
            System.out.println("Errore: " + e.getMessage());
        }
    }
}

```

Finally

Con la parola chiave **finally** possiamo eseguire un certo blocco di codice dopo il blocco try...catch, a prescindere dal fatto che si sia verificato o meno un errore.

```

public class Main {
    public static void main(String[] args) {
        try{
            int[] myNumbers = {1, 2, 3};
            System.out.println(myNumbers[10]);
        } catch (Exception e) {
            System.out.println("Errore: " + e.getMessage());
        } finally{
            //istruzioni eseguite a prescindere dal risultato
        }
    }
}

```

Throw

La parola chiave **throw** viene utilizzata per lanciare una eccezione Java.

Esempio:

```

public class Main {
    static void checkAge(int age) {
        if (age < 18) {
            throw new ArithmeticException("Accesso negato: devi avere almeno 18 anni per entrare!");
        }
        else {
            System.out.println("Accesso consentito, sei grande abbastanza");
        }
    }
}

```

Esistono numerosi tipi di eccezione. La seguente tabella evidenzia i più comuni.

Errore/Eccezione	Descrizione
ArithmeticError	Un calcolo matematico risulta in un errore
ArrayIndexOutOfBoundsException	Si cerca di accedere ad un array con un indice non valido
ClassNotFoundException	Si cerca di accedere ad una classe che non esiste
FileNotFoundException	Si cerca di accedere ad un file che non esiste
InputMismatchException	Si fornisce un input di tipo non corretto
IOException	Una operazione di I/O fallisce
NullPointerException	Si cerca di accedere ad un oggetto che è <i>null</i>
NumberFormatException	Quando non si riesce a convertire una stringa in un tipo numerico
StringIndexOutOfBoundsException	Si cerca di accedere ad un carattere di una stringa che non esiste